

SP-4 Phase A + B Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Land generic Credentials data model with per-user-passphrase AES-GCM encryption + per-provider configuration screen reachable from Menu (with credentials assignment, mirrors list/add/probe, sync toggle, clone, anonymous toggle).

Architecture: Phase A adds a provider-agnostic CredentialsEntry model (any of 4 secret types), Room tables for credentials + provider→credentials binding, and a PBKDF2/AES-GCM crypto chain with verifier-based wrong-passphrase rejection. Phase B adds a new :feature:provider_config Compose module reachable via Menu tap on a provider row; it composes credential assignment, mirror list/add/probe, sync ON/OFF toggle, anonymous toggle, and clone-provider into a single screen. Sync writes queue to a new SyncOutbox (Phase E uploader is honest-stub for now).

Tech Stack: Kotlin, Jetpack Compose, Orbit MVI, Room (KSP), Hilt, javax.crypto (AES-GCM + PBKDF2WithHmacSHA256), kotlinx-coroutines, kotlinx-serialization, JUnit 4 + mockk + MockWebServer + Compose UI Test.

File Structure

New files:

- core/credentials/src/main/kotlin/lava/credentials/model/CredentialsEntry.kt — CredentialsEntry data class.
- core/credentials/src/main/kotlin/lava/credentials/model/CredentialSecret.kt — sealed interface + CredentialType enum.

- core/credentials/src/main/kotlin/lava/credentials/crypto/CredentialsCrypto.kt — PBKDF2 + AES-GCM + verifier.
- core/credentials/src/main/kotlin/lava/credentials/CredentialsEntryRepository.kt — interface.
- core/credentials/src/main/kotlin/lava/credentials/CredentialsEntryRepositoryImpl.kt — impl.
- core/credentials/src/main/kotlin/lava/credentials/ProviderCredentialBinding.kt — interface + impl.
- core/database/src/main/kotlin/lava/database/entity/CredentialsEntryEntity.kt
- core/database/src/main/kotlin/lava/database/entity/ProviderCredentialBindingEntity.kt
- core/database/src/main/kotlin/lava/database/entity/ProviderSyncToggleEntity.kt
- core/database/src/main/kotlin/lava/database/entity/ClonedProviderEntity.kt
- core/database/src/main/kotlin/lava/database/entity/SyncOutboxEntity.kt
- core/database/src/main/kotlin/lava/database/dao/CredentialsEntryDao.kt
- core/database/src/main/kotlin/lava/database/dao/ProviderCredentialBindingDao.kt
- core/database/src/main/kotlin/lava/database/dao/ProviderSyncToggleDao.kt
- core/database/src/main/kotlin/lava/database/dao/ClonedProviderDao.kt
- core/database/src/main/kotlin/lava/database/dao/SyncOutboxDao.kt
- core/sync/build.gradle.kts + src/main/kotlin/lava/sync/SyncOutbox.kt + SyncOutboxImpl.kt.
- feature/credentials_manager/build.gradle.kts + src/main/kotlin/lava/credentials_manager/ (Screen, ViewModel, State, Action, SideEffect, Navigation.kt).
- feature/provider_config/build.gradle.kts + src/main/kotlin/lava/provider_config/ (Screen, ViewModel, State, Action, SideEffect, Navigation.kt, sections/*.kt).
- core/domain/src/main/kotlin/lava/domain/usecase/ProbeMirrorUseCase.kt
- core/domain/src/main/kotlin/lava/domain/usecase/CloneProviderUseCase.kt
- Test files: see per-task entries.
- Compose UI Challenges: Challenge27 through Challenge31 under app/src/androidTest/kotlin/lava/app/challenges/.

Modified files:

- `core/database/src/main/kotlin/lava/database/AppDatabase.kt:58` — bump version = 8 → version = 9; register 5 new entities + DAOs; add `MIGRATION_8_9`.
 - `core/data/src/main/kotlin/lava/data/converters/Endpoint.kt` — no changes (`Endpoint.Rutracker` filter already in place).
 - `feature/menu/src/main/kotlin/lava/menu/MenuScreen.kt:383-424` — `ProviderRow` becomes clickable → routes to `openProviderConfig(providerId)`; sign-out icon kept as trailing affordance.
 - `feature/menu/src/main/kotlin/lava/menu/MenuScreen.kt:204` — "Provider Credentials" entry now navigates to the new `:feature:credentials_manager`.
 - `app/src/main/kotlin/digital/vasic/lava/client/navigation/MobileNavigation.kt` — register both new feature nav extensions.
 - `core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt` — extend `listAvailableTrackers()` to union cloned descriptors from `ClonedProviderDao`.
 - `app/build.gradle.kts:28-29` — `versionCode 1036` → `1037`, `versionName "1.2.16"` → `"1.2.17"`.
 - `lava-api-go/internal/version/version.go` — `2.3.5/2305` → `2.3.6/2306`.
 - `CHANGELOG.md` — new entry for `Lava-Android-1.2.17-1037 / Lava-API-Go-2.3.6-2306`.
 - `docs/CONTINUATION.md §0` — reflect Phase A+B complete.
 - `.lava-ci-evidence/distribute-changelog/firebase-app-distribution/1.2.17-1037.md` — new snapshot.
-

Task 1: Domain model (`CredentialsEntry`, `CredentialSecret`, `CredentialType`)

Files:

- Create: `core/credentials/src/main/kotlin/lava/credentials/model/CredentialSecret.kt`
- Create: `core/credentials/src/main/kotlin/lava/credentials/model/CredentialsEntry.kt`



Step 1: Write the secret sealed-interface file

```
// core/credentials/src/main/kotlin/lava/credentials/model/
// CredentialSecret.kt
package lava.credentials.model

sealed interface CredentialSecret {
    data class UsernamePassword(val username: String, val
        password: String) : CredentialSecret
    data class ApiKey(val key: String) : CredentialSecret
    data class BearerToken(val token: String) : CredentialSecret
    data class CookieSession(val cookie: String) :
        CredentialSecret
}

enum class CredentialType { USERNAME_PASSWORD, API_KEY,
    BEARER_TOKEN, COOKIE_SESSION }

internal fun CredentialSecret.type(): CredentialType = when
    (this) {
        is CredentialSecret.UsernamePassword ->
            CredentialType.USERNAME_PASSWORD
        is CredentialSecret.ApiKey -> CredentialType.API_KEY
        is CredentialSecret.BearerToken -> CredentialType.BEARER_TOKEN
        is CredentialSecret.CookieSession ->
            CredentialType.COOKIE_SESSION
    }
}
```



Step 2: Write the entry data class

```
// core/credentials/src/main/kotlin/lava/credentials/model/
// CredentialsEntry.kt
package lava.credentials.model

data class CredentialsEntry(
    val id: String,
    val displayName: String,
    val type: CredentialType,
    val secret: CredentialSecret,
    val createdAtUtc: Long,
    val updatedAtUtc: Long,
)
```



Step 3: Compile

Run: ./gradlew :core:credentials:compileDebugKotlin Expected:
BUILD SUCCESSFUL



Step 4: Commit

```
git add core/credentials/src/main/kotlin/lava/credentials/model/
git commit -m "feat(credentials): SP-4 Phase A domain model –
    CredentialsEntry + sealed CredentialSecret +
    CredentialType"
```

Task 2: CredentialsCrypto (PBKDF2 + AES-GCM + verifier)

Files:

- Create: core/credentials/src/main/kotlin/lava/credentials/crypto/CredentialsCrypto.kt
- Test: core/credentials/src/test/kotlin/lava/credentials/crypto/CredentialsCryptoTest.kt



Step 1: Write the failing test

```
// core/credentials/src/test/kotlin/lava/credentials/crypto/
// CredentialsCryptoTest.kt
package lava.credentials.crypto

import org.junit.Assert.assertArrayEquals
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertTrue
import org.junit.Assert.fail
import org.junit.Test

class CredentialsCryptoTest {

    private val salt = ByteArray(32) { it.toByte() }
    private val passphrase = "correct horse battery staple"

    @Test
    fun `derive deterministic from same passphrase and salt`() {
        val k1 = CredentialsCrypto.deriveKey(passphrase, salt)
        val k2 = CredentialsCrypto.deriveKey(passphrase, salt)
        assertArrayEquals(k1, k2)
        assertEquals(32, k1.size)
    }

    @Test
    fun `verifier round trips and rejects wrong passphrase`() {
        val key = CredentialsCrypto.deriveKey(passphrase, salt)
        val verifier = CredentialsCrypto.makeVerifier(key)
        assertTrue(CredentialsCrypto.checkVerifier(key, verifier))
        val wrong = CredentialsCrypto.deriveKey("wrong
        passphrase", salt)
```

```

        assertFalse(CredentialsCrypto.checkVerifier(wrong,
            verifier))
    }

    @Test
    fun `encrypt then decrypt round trips`() {
        val key = CredentialsCrypto.deriveKey(passphrase, salt)
        val plain = "hunter2".toByteArray(Charsets.UTF_8)
        val ct = CredentialsCrypto.encrypt(key, plain)
        val pt = CredentialsCrypto.decrypt(key, ct)
        assertEquals(plain, pt)
    }

    @Test
    fun `decrypt with wrong key throws AEADBadTagException`() {
        val key = CredentialsCrypto.deriveKey(passphrase, salt)
        val wrong = CredentialsCrypto.deriveKey("wrong", salt)
        val ct = CredentialsCrypto.encrypt(key,
            "hunter2".toByteArray())
        try {
            CredentialsCrypto.decrypt(wrong, ct)
            fail("expected AEADBadTagException")
        } catch (e: javax.crypto.AEADBadTagException) {
            assertNotNull(e)
        }
    }

    @Test
    fun `tampered ciphertext throws AEADBadTagException`() {
        val key = CredentialsCrypto.deriveKey(passphrase, salt)
        val ct = CredentialsCrypto.encrypt(key,
            "hunter2".toByteArray())
        // Flip a byte in the ciphertext region (skip the 12-byte
        // nonce prefix).
        ct[20] = ct[20].inc()
        try {
            CredentialsCrypto.decrypt(key, ct)
            fail("expected AEADBadTagException")
        } catch (e: javax.crypto.AEADBadTagException) {
            assertNotNull(e)
        }
    }
}

```



Step 2: Run the test — expect compile failure

Run: `./gradlew :core:credentials:testDebugUnitTest --tests "lava.credentials.crypto.CredentialsCryptoTest"` Expected: FAIL — unresolved reference CredentialsCrypto.



Step 3: Write the implementation

```
// core/credentials/src/main/kotlin/lava/credentials/crypto/
// CredentialsCrypto.kt
package lava.credentials.crypto

import java.security.SecureRandom
import javax.crypto.Cipher
import javax.crypto.Mac
import javax.crypto.SecretKeyFactory
import javax.crypto.spec.GCMParameterSpec
import javax.crypto.spec.PBEKeySpec
import javax.crypto.spec.SecretKeySpec

/**
 * PBKDF2-SHA256 key derivation + AES-256-GCM encryption + HMAC-
 * SHA256
 * verifier for the SP-4 Phase A credentials chain. Key never
 * persists.
 */
object CredentialsCrypto {

    private const val PBKDF2_ITER = 200_000
    private const val KEY_BITS = 256
    private const val NONCE_BYTES = 12
    private const val TAG_BITS = 128
    private const val VERIFIER_TAG = "lava-credentials-v1"
    private val rng = SecureRandom()

    fun newSalt(): ByteArray = ByteArray(32).also {
        rng.nextBytes(it)
    }

    fun deriveKey(passphrase: String, salt: ByteArray): ByteArray {
        {
            val spec = PBEKeySpec(passphrase.toCharArray(), salt,
                PBKDF2_ITER, KEY_BITS)
            val factory =
                SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256")
            return factory.generateSecret(spec).encoded
        }
    }

    fun makeVerifier(key: ByteArray): ByteArray {
        val mac = Mac.getInstance("HmacSHA256")
        mac.init(SecretKeySpec(key, "HmacSHA256"))
        return
            mac.doFinal(VERIFIER_TAG.toByteArray(Charsets.UTF_8))
    }

    fun checkVerifier(key: ByteArray, expected: ByteArray):
        Boolean {
        val computed = makeVerifier(key)
        if (computed.size != expected.size) return false
        var diff = 0
    }
```

```

        for (i in computed.indices) diff = diff or
            (computed[i].toInt() xor expected[i].toInt())
        return diff == 0
    }

    fun encrypt(key: ByteArray, plaintext: ByteArray): ByteArray {
        val nonce = ByteArray(NONCE_BYTES).also {
            rng.nextBytes(it)
        }
        val cipher = Cipher.getInstance("AES/GCM/NoPadding")
        cipher.init(Cipher.ENCRYPT_MODE, SecretKeySpec(key,
            "AES"), GCMParameterSpec(TAG_BITS, nonce))
        val ct = cipher.doFinal(plaintext)
        return nonce + ct
    }

    fun decrypt(key: ByteArray, payload: ByteArray): ByteArray {
        require(payload.size > NONCE_BYTES + TAG_BITS / 8)
        val nonce = payload.copyOfRange(0, NONCE_BYTES)
        val ct = payload.copyOfRange(NONCE_BYTES, payload.size)
        val cipher = Cipher.getInstance("AES/GCM/NoPadding")
        cipher.init(Cipher.DECRYPT_MODE, SecretKeySpec(key,
            "AES"), GCMParameterSpec(TAG_BITS, nonce))
        return cipher.doFinal(ct)
    }
}

```



Step 4: Run the test — expect PASS

Run: `./gradlew :core:credentials:testDebugUnitTest --tests "lava.credentials.crypto.CredentialsCryptoTest"` Expected: 5 tests completed, all pass.



Step 5: Falsifiability rehearsal — record in commit

Mutate `decrypt()` to skip the `init` line (remove the `cipher.init(...)` call). Re-run tests. Expected fail: `decrypt` then `encrypt` round trips and tampering tests fail because the cipher isn't initialized. Revert.



Step 6: Commit

```

git add core/credentials/src/main/kotlin/lava/credentials/crypto/
    CredentialsCrypto.kt \
    core/credentials/src/test/kotlin/lava/credentials/crypto/
    CredentialsCryptoTest.kt
git commit -m "feat(credentials): SP-4 Phase A – PBKDF2 + AES-GCM
    + verifier crypto

```

Bluff-Audit: `core/credentials/src/test/kotlin/lava/credentials/`
`crypto/CredentialsCryptoTest.kt`

Mutation: remove cipher.init() call from
CredentialsCrypto.decrypt().
Observed-Failure: round-trip + tamper tests throw
IllegalStateException (Cipher not initialized) instead of
completing decrypt OR throwing AEADBadTagException.
Reverted: yes"

Task 3: Room entities + DAOs (5 new tables)

Files:

- Create: core/database/src/main/kotlin/lava/database/entity/CredentialsEntryEntity.kt
- Create: core/database/src/main/kotlin/lava/database/entity/ProviderCredentialBindingEntity.kt
- Create: core/database/src/main/kotlin/lava/database/entity/ProviderSyncToggleEntity.kt
- Create: core/database/src/main/kotlin/lava/database/entity/ClonedProviderEntity.kt
- Create: core/database/src/main/kotlin/lava/database/entity/SyncOutboxEntity.kt
- Create: core/database/src/main/kotlin/lava/database/dao/CredentialsEntryDao.kt
- Create: core/database/src/main/kotlin/lava/database/dao/ProviderCredentialBindingDao.kt
- Create: core/database/src/main/kotlin/lava/database/dao/ProviderSyncToggleDao.kt
- Create: core/database/src/main/kotlin/lava/database/dao/ClonedProviderDao.kt
- Create: core/database/src/main/kotlin/lava/database/dao/SyncOutboxDao.kt



Step 1: Write CredentialsEntryEntity

```
// core/database/src/main/kotlin/lava/database/entity/  
    CredentialsEntryEntity.kt  
package lava.database.entity  
  
import androidx.room.Entity  
import androidx.room.PrimaryKey  
  
@Entity(tableName = "credentials_entry")  
data class CredentialsEntryEntity(  
    @PrimaryKey val id: String,  
    val displayName: String,  
    val type: String,           // CredentialType.name
```

```
    val ciphertext: ByteArray,    // 12-byte nonce + GCM ct + tag
    val createdAt: Long,
    val updatedAt: Long,
)
```



Step 2: Write ProviderCredentialBindingEntity

```
// core/database/src/main/kotlin/lava/database/entity/
    ProviderCredentialBindingEntity.kt
package lava.database.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "provider_credential_binding")
data class ProviderCredentialBindingEntity(
    @PrimaryKey val providerId: String,
    val credentialId: String,
)
```



Step 3: Write ProviderSyncToggleEntity

```
// core/database/src/main/kotlin/lava/database/entity/
    ProviderSyncToggleEntity.kt
package lava.database.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "provider_sync_toggle")
data class ProviderSyncToggleEntity(
    @PrimaryKey val providerId: String,
    val enabled: Boolean,
)
```



Step 4: Write ClonedProviderEntity

```
// core/database/src/main/kotlin/lava/database/entity/
    ClonedProviderEntity.kt
package lava.database.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "cloned_provider")
data class ClonedProviderEntity(
    @PrimaryKey val syntheticId: String,
    val sourceTrackerId: String,
    val displayName: String,
)
```

```
        val primaryUrL: String,
    )
```



Step 5: Write SyncOutboxEntity

```
// core/database/src/main/kotlin/lava/database/entity/
    SyncOutboxEntity.kt
package lava.database.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "sync_outbox")
data class SyncOutboxEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val kind: String,           // "credentials" | "binding" |
        "sync_toggle" | "cloned_provider" | "user_mirror"
    val payload: String,       // JSON
    val createdAt: Long,
)
```



Step 6: Write all 5 DAOs

```
// core/database/src/main/kotlin/lava/database/dao/
    CredentialsEntryDao.kt
package lava.database.dao

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import kotlinx.coroutines.flow.Flow
import lava.database.entity.CredentialsEntryEntity

@Dao
interface CredentialsEntryDao {
    @Query("SELECT * FROM credentials_entry ORDER BY updatedAt
        DESC")
    fun observeAll(): Flow<List<CredentialsEntryEntity>>

    @Query("SELECT * FROM credentials_entry WHERE id = :id")
    suspend fun get(id: String): CredentialsEntryEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(entity: CredentialsEntryEntity)

    @Query("DELETE FROM credentials_entry WHERE id = :id")
    suspend fun delete(id: String)
}
```

```

// core/database/src/main/kotlin/lava/database/dao/
    ProviderCredentialBindingDao.kt
package lava.database.dao

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import kotlinx.coroutines.flow.Flow
import lava.database.entity.ProviderCredentialBindingEntity

@Dao
interface ProviderCredentialBindingDao {
    @Query("SELECT * FROM provider_credential_binding WHERE
        providerId = :providerId")
    fun observe(providerId: String):
        Flow<ProviderCredentialBindingEntity?>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(entity: ProviderCredentialBindingEntity)

    @Query("DELETE FROM provider_credential_binding WHERE
        providerId = :providerId")
    suspend fun unbind(providerId: String)
}

// core/database/src/main/kotlin/lava/database/dao/
    ProviderSyncToggleDao.kt
package lava.database.dao

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import kotlinx.coroutines.flow.Flow
import lava.database.entity.ProviderSyncToggleEntity

@Dao
interface ProviderSyncToggleDao {
    @Query("SELECT * FROM provider_sync_toggle WHERE providerId
        = :providerId")
    fun observe(providerId: String):
        Flow<ProviderSyncToggleEntity?>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(entity: ProviderSyncToggleEntity)
}

// core/database/src/main/kotlin/lava/database/dao/
    ClonedProviderDao.kt
package lava.database.dao

import androidx.room.Dao

```

```

import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import kotlinx.coroutines.flow.Flow
import lava.database.entity.ClonedProviderEntity

@Dao
interface ClonedProviderDao {
    @Query("SELECT * FROM cloned_provider")
    fun observeAll(): Flow<List<ClonedProviderEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun upsert(entity: ClonedProviderEntity)

    @Query("DELETE FROM cloned_provider WHERE syntheticId = :id")
    suspend fun delete(id: String)
}

// core/database/src/main/kotlin/lava/database/dao/
// SyncOutboxDao.kt
package lava.database.dao

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query
import kotlinx.coroutines.flow.Flow
import lava.database.entity.SyncOutboxEntity

@Dao
interface SyncOutboxDao {
    @Insert
    suspend fun enqueue(entity: SyncOutboxEntity): Long

    @Query("SELECT * FROM sync_outbox ORDER BY createdAt ASC")
    fun observeAll(): Flow<List<SyncOutboxEntity>>

    @Query("DELETE FROM sync_outbox WHERE id = :id")
    suspend fun ack(id: Long)
}

```



Step 7: Compile

Run: ./gradlew :core:database:compileDebugKotlin Expected: BUILD SUCCESSFUL.



Step 8: Commit

```
git add core/database/src/main/kotlin/lava/database/entity/
      {Credentials,ProviderCredentialBinding,ProviderSyncToggle,ClonedProvider,S
      *.kt \
      core/database/src/main/kotlin/lava/database/dao/
      {CredentialsEntry,ProviderCredentialBinding,ProviderSyncToggle,ClonedProv
      Dao.kt
git commit -m "feat(database): SP-4 Phase A+B – 5 new entities +
      DAOs"
```

Task 4: AppDatabase migration 8 → 9 + DAO accessors

Files:

- Modify: core/database/src/main/kotlin/lava/database/AppDatabase.kt



Step 1: Add entities to the @Database(entities = [...]) list

In AppDatabase.kt, insert into the entities array (alphabetically by class name):

```
ClonedProviderEntity::class,
CredentialsEntryEntity::class,
ProviderCredentialBindingEntity::class,
ProviderSyncToggleEntity::class,
SyncOutboxEntity::class,
```



Step 2: Bump version

Change version = 8, to version = 9,.



Step 3: Add DAO accessors

After the existing abstract fun forumProviderSelectionDao(), append:

```
abstract fun credentialsEntryDao(): CredentialsEntryDao
abstract fun providerCredentialBindingDao():
      ProviderCredentialBindingDao
abstract fun providerSyncToggleDao(): ProviderSyncToggleDao
abstract fun clonedProviderDao(): ClonedProviderDao
abstract fun syncOutboxDao(): SyncOutboxDao
```



Step 4: Write the migration

Inside the companion Migration_3_4 block, append after the last existing migration:

```
val MIGRATION_8_9 = object : Migration(8, 9) {
    override fun migrate(db: SupportSQLiteDatabase) {
        db.execSQL(
            "CREATE TABLE IF NOT EXISTS credentials_entry (" +
                "id TEXT NOT NULL PRIMARY KEY, displayName TEXT " +
                "NOT NULL, type TEXT NOT NULL, " +
                "ciphertext BLOB NOT NULL, createdAt INTEGER NOT " +
                "NULL, updatedAt INTEGER NOT NULL)"
        )
        db.execSQL(
            "CREATE TABLE IF NOT EXISTS " +
            provider_credential_binding (" +
                "providerId TEXT NOT NULL PRIMARY KEY, " +
                "credentialId TEXT NOT NULL)"
        )
        db.execSQL(
            "CREATE TABLE IF NOT EXISTS provider_sync_toggle (" +
                "providerId TEXT NOT NULL PRIMARY KEY, enabled " +
                "INTEGER NOT NULL)"
        )
        db.execSQL(
            "CREATE TABLE IF NOT EXISTS cloned_provider (" +
                "syntheticId TEXT NOT NULL PRIMARY KEY, " +
                "sourceTrackerId TEXT NOT NULL, " +
                "displayName TEXT NOT NULL, primaryUri TEXT NOT " +
                "NULL)"
        )
        db.execSQL(
            "CREATE TABLE IF NOT EXISTS sync_outbox (" +
                "id INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT, " +
                "kind TEXT NOT NULL, " +
                "payload TEXT NOT NULL, createdAt INTEGER NOT " +
                "NULL)"
        )
    }
}
```



Step 5: Wire migration into the database builder

Find the Migration_** registration block (typically in the database provider; search for addMigrations()). Add MIGRATION_8_9 to the list.



Step 6: Compile + schema export

Run: ./gradlew :core:database:compileDebugKotlin Expected: BUILD SUCCESSFUL. Verify core/database/schemas/lava.database.AppDatabase/9.json exists post-build.



Step 7: Commit

```
git add core/database/src/main/kotlin/lava/database/
      AppDatabase.kt \
      core/database/schemas/lava.database.AppDatabase/9.json
git commit -m "feat(database): SP-4 Phase A+B – bump schema v8 →
      v9; MIGRATION_8_9"
```

Task 5: CredentialsEntryRepository (interface + impl)

Files:

- Create: core/credentials/src/main/kotlin/lava/credentials/CredentialsEntryRepository.kt
- Create: core/credentials/src/main/kotlin/lava/credentials/CredentialsEntryRepositoryImpl.kt
- Test: core/credentials/src/test/kotlin/lava/credentials/CredentialsEntryRepositoryImplTest.kt



Step 1: Write the test (real DAO via in-memory Room is overkill — fake DAO)

```
// core/credentials/src/test/kotlin/lava/credentials/
      CredentialsEntryRepositoryImplTest.kt
package lava.credentials

import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.runBlocking
import lava.credentials.crypto.CredentialsCrypto
import lava.credentials.model.CredentialSecret
import lava.credentials.model.CredentialType
import lava.credentials.model.CredentialsEntry
import lava.database.dao.CredentialsEntryDao
import lava.database.entity.CredentialsEntryEntity
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNotNull
import org.junit.Assert.assertNull
import org.junit.Test

class CredentialsEntryRepositoryImplTest {
    private class FakeDao : CredentialsEntryDao {
```

```

    val flow =
        MutableStateFlow<List<CredentialsEntryEntity>>(emptyList())
    override fun observeAll() = flow
    override suspend fun get(id: String) =
        flow.value.firstOrNull { it.id == id }
    override suspend fun upsert(entity:
        CredentialsEntryEntity) {
        flow.value = (flow.value.filterNot { it.id ==
            entity.id } + entity)
    }
    override suspend fun delete(id: String) { flow.value =
        flow.value.filterNot { it.id == id } }
}

```

```

private val salt = ByteArray(32) { it.toByte() }
private val key = CredentialsCrypto.deriveKey("pass", salt)

```

```

@Test
fun `upsert encrypts and decrypts on read`() = runBlocking {
    val dao = FakeDao()
    val repo = CredentialsEntryRepositoryImpl(dao) { key }
    val entry = CredentialsEntry(
        id = "id-1",
        displayName = "My creds",
        type = CredentialType.USERNAME_PASSWORD,
        secret = CredentialSecret.UsernamePassword("alice",
            "p"),
        createdAtUtc = 1,
        updatedAtUtc = 2,
    )
    repo.upsert(entry)
    val read = repo.list().first()
    assertEquals(1, read.size)
    assertEquals("My creds", read[0].displayName)
    assertEquals(CredentialSecret.UsernamePassword("alice",
        "p"), read[0].secret)
}

```

```

@Test
fun `get returns null for unknown id`() = runBlocking {
    val dao = FakeDao()
    val repo = CredentialsEntryRepositoryImpl(dao) { key }
    assertNull(repo.get("nope"))
}

```

```

@Test
fun `delete removes the row`() = runBlocking {
    val dao = FakeDao()
    val repo = CredentialsEntryRepositoryImpl(dao) { key }
    val entry = CredentialsEntry("id-2", "X",
        CredentialType.API_KEY,
        CredentialSecret.ApiKey("k"), 1, 2)
    repo.upsert(entry)
}

```

```

        assertNotNull(repo.get("id-2"))
        repo.delete("id-2")
        assertNull(repo.get("id-2"))
    }
}

```



Step 2: Run test → expect compile fail (impl missing)

Run: `./gradlew :core:credentials:testDebugUnitTest --tests "*CredentialsEntryRepositoryImplTest*" Expected: FAIL with unresolved references.`



Step 3: Write the interface

```

// core/credentials/src/main/kotlin/lava/credentials/
//   CredentialsEntryRepository.kt
package lava.credentials

import kotlinx.coroutines.flow.Flow
import lava.credentials.model.CredentialsEntry

interface CredentialsEntryRepository {
    fun observe(): Flow<List<CredentialsEntry>>
    suspend fun list(): List<CredentialsEntry>
    suspend fun get(id: String): CredentialsEntry?
    suspend fun upsert(entry: CredentialsEntry)
    suspend fun delete(id: String)
}

```



Step 4: Write the impl

```

// core/credentials/src/main/kotlin/lava/credentials/
//   CredentialsEntryRepositoryImpl.kt
package lava.credentials

import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.flow.map
import kotlinx.serialization.json.Json
import kotlinx.serialization.encodeToString
import kotlinx.serialization.decodeFromString
import lava.credentials.crypto.CredentialsCrypto
import lava.credentials.model.CredentialSecret
import lava.credentials.model.CredentialType
import lava.credentials.model.CredentialsEntry
import lava.database.dao.CredentialsEntryDao
import lava.database.entity.CredentialsEntryEntity
import kotlinx.serialization.Serializable

```

```

import javax.inject.Inject
import javax.inject.Singleton

private typealias KeyProvider = () -> ByteArray

@Singleton
class CredentialsEntryRepositoryImpl @Inject constructor(
    private val dao: CredentialsEntryDao,
    private val keyProvider: KeyProvider,
) : CredentialsEntryRepository {

    override fun observe(): Flow<List<CredentialsEntry>> =
        dao.observeAll().map { rows -> rows.map(::decode) }

    override suspend fun list(): List<CredentialsEntry> =
        observe().first()

    override suspend fun get(id: String): CredentialsEntry? =
        dao.get(id)?.let(::decode)

    override suspend fun upsert(entry: CredentialsEntry) {
        val payload = json.encodeToString(entry.secret.toWire())
        val ct = CredentialsCrypto.encrypt(keyProvider(),
            payload.toByteArray(Charsets.UTF_8))
        dao.upsert(
            CredentialsEntryEntity(
                id = entry.id,
                displayName = entry.displayName,
                type = entry.type.name,
                ciphertext = ct,
                createdAt = entry.createdAtUtc,
                updatedAt = entry.updatedAtUtc,
            ),
        )
    }

    override suspend fun delete(id: String) = dao.delete(id)

    private fun decode(entity: CredentialsEntryEntity):
        CredentialsEntry {
        val pt = CredentialsCrypto.decrypt(keyProvider(),
            entity.ciphertext).toString(Charsets.UTF_8)
        val secret =
            json.decodeFromString<WireSecret>(pt).toDomain()
        return CredentialsEntry(
            id = entity.id,
            displayName = entity.displayName,
            type = CredentialType.valueOf(entity.type),
            secret = secret,
            createdAtUtc = entity.createdAt,
            updatedAtUtc = entity.updatedAt,
        )
    }
}

```

```

@Serializable
private data class WireSecret(
    val kind: String,
    val a: String = "",
    val b: String = "",
) {
    fun toDomain(): CredentialSecret = when (kind) {
        "up" -> CredentialSecret.UsernamePassword(a, b)
        "ak" -> CredentialSecret.ApiKey(a)
        "bt" -> CredentialSecret.BearerToken(a)
        "cs" -> CredentialSecret.CookieSession(a)
        else -> error("unknown secret kind: $kind")
    }
}

private fun CredentialSecret.toWire(): WireSecret = when
    (this) {
        is CredentialSecret.UsernamePassword -> WireSecret("up",
            username, password)
        is CredentialSecret.ApiKey -> WireSecret("ak", key)
        is CredentialSecret.BearerToken -> WireSecret("bt", token)
        is CredentialSecret.CookieSession -> WireSecret("cs",
            cookie)
    }

companion object {
    private val json = Json { ignoreUnknownKeys = true }
}

```



Step 5: Run tests → expect PASS

Run: `./gradlew :core:credentials:testDebugUnitTest --tests
 "*CredentialsEntryRepositoryImplTest*" Expected: 3 tests PASS.`



Step 6: Falsifiability rehearsal

Mutate `upsert()` to skip `credentialsCrypto.encrypt(...)` and store `payload.toByteArray(...)` directly (no encryption). Re-run tests. Expected fail: `decode()` reads the unencrypted bytes as ciphertext + nonce, throws either `AEADBadTagException` or `BufferOverflow`. Revert.



Step 7: Commit

```
git add core/credentials/src/main/kotlin/lava/credentials/
    CredentialsEntryRepository*.kt \
    core/credentials/src/test/kotlin/lava/credentials/
    CredentialsEntryRepositoryImplTest.kt
git commit -m "feat(credentials): SP-4 Phase A –
    CredentialsEntryRepository{,Impl} + tests

Bluff-Audit: core/credentials/.../
    CredentialsEntryRepositoryImplTest.kt
Mutation: stored plaintext via dao.upsert(...
    payload.toByteArray() ...) skipping the encrypt() call.
Observed-Failure: 'upsert encrypts and decrypts on read' fails
    because decode() invokes CredentialsCrypto.decrypt() on
    plaintext bytes and throws either AEADBadTagException or
    IllegalArgumentException.
Reverted: yes"
```

Task 6: ProviderCredentialBinding (interface + impl)

Files:

- Create: core/credentials/src/main/kotlin/lava/credentials/ProviderCredentialBinding.kt
- Create: core/credentials/src/main/kotlin/lava/credentials/ProviderCredentialBindingImpl.kt
- Test: core/credentials/src/test/kotlin/lava/credentials/ProviderCredentialBindingImplTest.kt



Step 1: Write the test (mirrors Task 5 pattern; FakeDao stores ProviderCredentialBindingEntity rows; tests assert bind / unbind / observe).

package lava.credentials

```
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.runBlocking
import lava.database.dao.ProviderCredentialBindingDao
import lava.database.entity.ProviderCredentialBindingEntity
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNull
import org.junit.Test

class ProviderCredentialBindingImplTest {
    private class FakeDao : ProviderCredentialBindingDao {
        val rows = mutableMapOf<String,
            ProviderCredentialBindingEntity>()
        val flow =
            MutableStateFlow<List<ProviderCredentialBindingEntity>>(emptyList())
```

```

        override fun observe(providerId: String) =
            MutableStateFlow(rows[providerId])
        override suspend fun upsert(entity:
            ProviderCredentialBindingEntity) {
            rows[entity.providerId] = entity }
        override suspend fun unbind(providerId: String) {
            rows.remove(providerId) }
    }

    @Test
    fun `bind then observe returns the credential id`() =
        runBlocking {
            val dao = FakeDao()
            val binding = ProviderCredentialBindingImpl(dao)
            binding.bind("rutracker", "cred-1")
            assertEquals("cred-1",
                binding.observe("rutracker").first())
        }

    @Test
    fun `unbind clears the binding`() = runBlocking {
        val dao = FakeDao()
        val binding = ProviderCredentialBindingImpl(dao)
        binding.bind("rutor", "cred-1")
        binding.unbind("rutor")
        assertNull(binding.observe("rutor").first())
    }
}

```



Step 2: Write interface + impl

```

// core/credentials/src/main/kotlin/lava/credentials/
// ProviderCredentialBinding.kt
package lava.credentials

import kotlinx.coroutines.flow.Flow

interface ProviderCredentialBinding {
    suspend fun bind(providerId: String, credentialId: String)
    suspend fun unbind(providerId: String)
    fun observe(providerId: String): Flow<String?>
}

// core/credentials/src/main/kotlin/lava/credentials/
// ProviderCredentialBindingImpl.kt
package lava.credentials

import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import lava.database.dao.ProviderCredentialBindingDao
import lava.database.entity.ProviderCredentialBindingEntity
import javax.inject.Inject

```

```
import javax.inject.Singleton

@Singleton
class ProviderCredentialBindingImpl @Inject constructor(
    private val dao: ProviderCredentialBindingDao,
) : ProviderCredentialBinding {
    override suspend fun bind(providerId: String, credentialId:
        String) {
        dao.upsert(ProviderCredentialBindingEntity(providerId,
            credentialId))
    }
    override suspend fun unbind(providerId: String) =
        dao.unbind(providerId)
    override fun observe(providerId: String): Flow<String?> =
        dao.observe(providerId).map { it?.credentialId }
}
```



Step 3: Run tests Expected: 2 PASS.



Step 4: Falsifiability Mutate bind() to no-op; bind then observe returns the credential id fails. Revert.



Step 5: Commit with Bluff-Audit stamp.

Task 7: Passphrase session cache + key provider

Files:

- Create: core/credentials/src/main/kotlin/lava/credentials/session/CredentialsKeyHolder.kt
- Test: core/credentials/src/test/kotlin/lava/credentials/session/CredentialsKeyHolderTest.kt



Step 1: Write the test

```
package lava.credentials.session
```

```
import org.junit.Assert.assertEquals
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNull
import org.junit.Assert.assertTrue
import org.junit.Test
```

```
class CredentialsKeyHolderTest {
    @Test fun `starts locked`() {
        val h = CredentialsKeyHolder()
```

```

        assertNull(h.getOrNull())
    }
    @Test fun `unlock then getOrNull returns same key`() {
        val h = CredentialsKeyHolder()
        val k = ByteArray(32) { 1 }
        h.unlock(k)
        assertEquals(k, h.getOrNull())
    }
    @Test fun `lock zeroes the key`() {
        val h = CredentialsKeyHolder()
        val k = ByteArray(32) { 7 }
        h.unlock(k)
        h.lock()
        assertNull(h.getOrNull())
        assertTrue(k.all { it == 0.toByte() })
    }
    @Test fun `isUnlocked reflects state`() {
        val h = CredentialsKeyHolder()
        assertFalse(h.isUnlocked())
        h.unlock(ByteArray(32))
        assertTrue(h.isUnlocked())
    }
}

```



Step 2: Write impl

```

// core/credentials/src/main/kotlin/lava/credentials/session/
//   CredentialsKeyHolder.kt
package lava.credentials.session

import java.util.concurrent.atomic.AtomicReference
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class CredentialsKeyHolder @Inject constructor() {
    private val ref = AtomicReference<ByteArray?>(null)

    fun unlock(key: ByteArray) { ref.set(key) }

    fun lock() {
        val k = ref.getAndSet(null)
        k?.fill(0)
    }

    fun isUnlocked(): Boolean = ref.get() != null

    fun getOrNull(): ByteArray? = ref.get()

    fun require(): ByteArray = ref.get()
}

```

```

        ?: error("credentials key holder is locked – prompt user
        for passphrase first")
    }
}

```



Step 3: Run + commit with Bluff-Audit (mutate `lock()` to skip the `fail(0)`; the zeroing test fails).

Task 8: PassphraseManager (persists salt + verifier; unlocks the KeyHolder)

Files:

- Create: `core/credentials/src/main/kotlin/lava/credentials/PassphraseManager.kt`
- Test: `core/credentials/src/test/kotlin/lava/credentials/PassphraseManagerTest.kt`



Step 1: Write the test — verify (a) `firstTimeSetup` writes salt + verifier; (b) unlock with correct passphrase succeeds + populates `KeyHolder`; (c) unlock with wrong passphrase returns false + `KeyHolder` stays locked.

```
package lava.credentials
```

```

import lava.credentials.session.CredentialsKeyHolder
import org.junit.Assert.assertFalse
import org.junit.Assert.assertTrue
import org.junit.Test
import kotlinx.coroutines.runBlocking

```

```

class PassphraseManagerTest {
    private class FakeStore : PassphraseManager.Storage {
        var salt: ByteArray? = null
        var verifier: ByteArray? = null
        override fun saveSalt(b: ByteArray) { salt = b }
        override fun getSalt(): ByteArray? = salt
        override fun saveVerifier(b: ByteArray) { verifier = b }
        override fun getVerifier(): ByteArray? = verifier
    }

    @Test fun `firstTimeSetup persists salt and verifier`() =
        runBlocking {
            val store = FakeStore()
            val holder = CredentialsKeyHolder()
            val m = PassphraseManager(store, holder)
            m.firstTimeSetup("pw")
        }
}

```

```

        assertTrue(store.salt != null && store.verifier != null)
        assertTrue(holder.isUnlocked())
    }

    @Test fun `unlock right passphrase succeeds`() = runBlocking {
        val store = FakeStore()
        val holder = CredentialsKeyHolder()
        val m = PassphraseManager(store, holder)
        m.firstTimeSetup("pw")
        holder.lock()
        assertTrue(m.unlock("pw"))
        assertTrue(holder.isUnlocked())
    }

    @Test fun `unlock wrong passphrase fails and stays locked`()
        = runBlocking {
        val store = FakeStore()
        val holder = CredentialsKeyHolder()
        val m = PassphraseManager(store, holder)
        m.firstTimeSetup("pw")
        holder.lock()
        assertFalse(m.unlock("nope"))
        assertFalse(holder.isUnlocked())
    }
}

```



Step 2: Write impl

```

// core/credentials/src/main/kotlin/lava/credentials/
//   PassphraseManager.kt
package lava.credentials

import lava.credentials.crypto.CredentialsCrypto
import lava.credentials.session.CredentialsKeyHolder
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class PassphraseManager @Inject constructor(
    private val storage: Storage,
    private val keyHolder: CredentialsKeyHolder,
) {
    interface Storage {
        fun saveSalt(b: ByteArray)
        fun getSalt(): ByteArray?
        fun saveVerifier(b: ByteArray)
        fun getVerifier(): ByteArray?
    }

    fun isInitialized(): Boolean = storage.getSalt() != null &&
        storage.getVerifier() != null
}

```

```

suspend fun firstTimeSetup(passphrase: String) {
    val salt = CredentialsCrypto.newSalt()
    val key = CredentialsCrypto.deriveKey(passphrase, salt)
    storage.saveSalt(salt)
    storage.saveVerifier(CredentialsCrypto.makeVerifier(key))
    keyHolder.unlock(key)
}

suspend fun unlock(passphrase: String): Boolean {
    val salt = storage.getSalt() ?: return false
    val verifier = storage.getVerifier() ?: return false
    val key = CredentialsCrypto.deriveKey(passphrase, salt)
    if (!CredentialsCrypto.checkVerifier(key, verifier))
        return false
    keyHolder.unlock(key)
    return true
}
}

```



Step 3: Add PassphraseManager.Storage impl backed by SharedPreferences in a separate file `core/credentials/src/main/kotlin/lava/credentials/SharedPrefsPassphraseStorage.kt` —
Base64 encode salt + verifier in the settings prefs file under keys `credentials_kdf_salt_v1` + `credentials_verifier_v1`.

```
package lava.credentials
```

```

import android.content.SharedPreferences
import android.util.Base64
import androidx.core.content.edit
import javax.inject.Inject

class SharedPrefsPassphraseStorage @Inject constructor(
    private val prefs: SharedPreferences,
) : PassphraseManager.Storage {
    override fun saveSalt(b: ByteArray) {
        prefs.edit { putString(SALT_KEY, Base64.encodeToString(b, Base64.NO_WRAP)) }
    }

    override fun getSalt(): ByteArray? =
        prefs.getString(SALT_KEY, null)?.let { Base64.decode(it, Base64.NO_WRAP) }

    override fun saveVerifier(b: ByteArray) {
        prefs.edit { putString(VERIFIER_KEY, Base64.encodeToString(b, Base64.NO_WRAP)) }
    }

    override fun getVerifier(): ByteArray? =
        prefs.getString(VERIFIER_KEY, null)?.let { Base64.decode(it, Base64.NO_WRAP) }
}

```

```

        private companion object {
            const val SALT_KEY = "credentials_kdf_salt_v1"
            const val VERIFIER_KEY = "credentials_verifier_v1"
        }
    }
}

```



Step 4: Run + commit with Bluff-Audit stamp.

Task 9: Hilt wiring for credentials module

Files:

- Modify: core/credentials/src/main/kotlin/lava/credentials/di/CredentialsModule.kt



Step 1: Add new bindings

```

// Append to existing CredentialsModule:
@Binds
@Singleton
abstract fun bindCredentialsEntryRepository(impl:
    CredentialsEntryRepositoryImpl):
    CredentialsEntryRepository

@Binds
@Singleton
abstract fun bindProviderCredentialBinding(impl:
    ProviderCredentialBindingImpl): ProviderCredentialBinding

@Binds
@Singleton
abstract fun bindPassphraseStorage(impl:
    SharedPrefsPassphraseStorage): PassphraseManager.Storage

@Module
@InstallIn(SingletonComponent::class)
object CredentialsKeyProviderModule {
    @Provides
    @Singleton
    fun keyProvider(holder: CredentialsKeyHolder): () ->
        ByteArray = { holder.require() }
}

```

(Also add the DAO providers if not already wired through the database module — usually :core:database already provides DAOs; otherwise add @Provides fun credentialsEntryDao(db: AppDatabase) = db.credentialsEntryDao().)



Step 2: Compile

Run: `./gradlew :core:credentials:assembleDebug`



Step 3: Commit

```
git commit -m "feat(credentials): SP-4 Phase A – Hilt wiring for  
new repo/binding/passphrase types"
```

Task 10: Credentials Manager feature module

Files:

- Create: `feature/credentials_manager/build.gradle.kts`
- Create: `feature/credentials_manager/src/main/AndroidManifest.xml`
- Create: `feature/credentials_manager/src/main/kotlin/lava/credentials_manager/{CredentialsManagerScreen,ViewModel,State,Action,SideEffect,Navigation}.kt`
- Modify: `settings.gradle.kts` (register new module)



Step 1: Add module declaration

```
// feature/credentials_manager/build.gradle.kts  
plugins {  
    id("lava.android.feature")  
}  
  
android {  
    namespace = "lava.credentials_manager"  
}  
  
dependencies {  
    implementation(project(":core:credentials"))  
    implementation(project(":core:designsystem"))  
    implementation(project(":core:navigation"))  
}  
  

```

Append `include(":feature:credentials_manager")` next to the other `:feature:*` includes.



Step 3: Write State + Action + SideEffect

// State.kt

```
package lava.credentials_manager
```

```
import lava.credentials.model.CredentialsEntry
```

```
data class CredentialsManagerState(  
    val unlocked: Boolean = false,  
    val initializing: Boolean = false,  
    val unlockError: String? = null,  
    val entries: List<CredentialsEntry> = emptyList(),  
    val editing: CredentialsEntry? = null,  
)
```

// Action.kt

```
package lava.credentials_manager
```

```
import lava.credentials.model.CredentialSecret
```

```
sealed interface CredentialsManagerAction {  
    data class Unlock(val passphrase: String) :  
        CredentialsManagerAction  
    data class FirstTimeSetup(val passphrase: String) :  
        CredentialsManagerAction  
    data object AddNew : CredentialsManagerAction  
    data class Edit(val id: String) : CredentialsManagerAction  
    data class Save(val displayName: String, val secret:  
        CredentialSecret) : CredentialsManagerAction  
    data class Delete(val id: String) : CredentialsManagerAction  
    data object DismissEdit : CredentialsManagerAction  
}
```

// SideEffect.kt

```
package lava.credentials_manager
```

```
sealed interface CredentialsManagerSideEffect {  
    data class ShowToast(val msg: String) :  
        CredentialsManagerSideEffect  
}
```



Step 4: Write the ViewModel

// ViewModel.kt

```
package lava.credentials_manager
```

```
import androidx.lifecycle.ViewModel
```

```

import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.collect
import lava.credentials.CredentialsEntryRepository
import lava.credentials.PassphraseManager
import lava.credentials.model.CredentialSecret
import lava.credentials.model.CredentialType
import lava.credentials.model.CredentialsEntry
import lava.credentials.session.CredentialsKeyHolder
import org.orbitmvi.orbit.ContainerHost
import org.orbitmvi.orbit.syntax.simple.intent
import org.orbitmvi.orbit.syntax.simple.postSideEffect
import org.orbitmvi.orbit.syntax.simple.reduce
import org.orbitmvi.orbit.viewmodel.container
import java.util.UUID
import javax.inject.Inject

@HiltViewModel
class CredentialsManagerViewModel @Inject constructor(
    private val repo: CredentialsEntryRepository,
    private val passphrase: PassphraseManager,
    private val keyHolder: CredentialsKeyHolder,
) : ContainerHost<CredentialsManagerState,
    CredentialsManagerSideEffect>, ViewModel() {

    override val container = container<CredentialsManagerState,
        CredentialsManagerSideEffect>(
        initialState = CredentialsManagerState(
            unlocked = keyHolder.isUnlocked(),
            initializing = !passphrase.isInitialized(),
        ),
        onCreate = { if (keyHolder.isUnlocked())
            observeEntries() },
    )

    fun perform(action: CredentialsManagerAction) = intent {
        when (action) {
            is CredentialsManagerAction.FirstTimeSetup -> {
                passphrase.firstTimeSetup(action.passphrase)
                reduce { state.copy(unlocked = true, initializing
                    = false) }
                observeEntries()
            }
            is CredentialsManagerAction.Unlock -> {
                if (passphrase.unlock(action.passphrase)) {
                    reduce { state.copy(unlocked = true,
                        unlockError = null) }
                    observeEntries()
                } else {
                    reduce { state.copy(unlockError = "Wrong
                        passphrase") }
                }
            }
        }
    }
}

```

```

        CredentialsManagerAction.AddNew -> reduce {
state.copy(editing = empty()) }
        is CredentialsManagerAction.Edit -> {
            val existing = repo.get(action.id) ?:
return@intent
            reduce { state.copy(editing = existing) }
        }
        is CredentialsManagerAction.Save -> {
            val now = System.currentTimeMillis()
            val current = state.editing ?: empty()
            val saved = current.copy(
                displayName = action.displayName,
                type = action.secret.type(),
                secret = action.secret,
                updatedAtUtc = now,
            )
            repo.upsert(saved)
            reduce { state.copy(editing = null) }

postSideEffect(CredentialsManagerSideEffect.ShowToast("Saved"))
        }
        is CredentialsManagerAction.Delete -> {
            repo.delete(action.id)

postSideEffect(CredentialsManagerSideEffect.ShowToast("Deleted"))
        }
        CredentialsManagerAction.DismissEdit -> reduce {
state.copy(editing = null) }
    }
}

private suspend fun
    org.orbitmvi.orbit.syntax.simple.SimpleSyntax<CredentialsManagerState,
CredentialsManagerSideEffect>.observeEntries() {
    repo.observe().collect { entries -> reduce {
state.copy(entries = entries) } }
}

private fun empty() = CredentialsEntry(
    id = UUID.randomUUID().toString(),
    displayName = "",
    type = CredentialType.USERNAME_PASSWORD,
    secret = CredentialSecret.UsernamePassword("", ""),
    createdAtUtc = System.currentTimeMillis(),
    updatedAtUtc = System.currentTimeMillis(),
)

private fun CredentialSecret.type() = when (this) {
    is CredentialSecret.UsernamePassword ->
        CredentialType.USERNAME_PASSWORD
    is CredentialSecret.ApiKey -> CredentialType.API_KEY
    is CredentialSecret.BearerToken ->
        CredentialType.BEARER_TOKEN
}

```

```

        is CredentialSecret.CookieSession ->
            CredentialType.COOKIE_SESSION
    }
}

```



Step 5: Write the Screen composable — passphrase unlock layer (renders when not unlocked), list of entries with overflow menu (edit / delete), FAB to add, edit sheet with type picker + per-type fields. Use existing `:core:designsystem` components.



Step 6: Write Navigation.kt — `addCredentialsManager()` + `openCredentialsManager()` per `:core:navigation` DSL pattern (mirror existing feature modules).



Step 7: Register navigation in `MobileNavigation.kt` (`addCredentialsManager()` inside the graph).



Step 8: Compile → `./gradlew :feature:credentials_manager:assembleDebug`.



Step 9: Commit with message describing the Phase A UI scaffold.

Task 11: Compose UI Challenge — Challenge27 + Challenge31

Files:

- Create: `app/src/androidTest/kotlin/lava/app/challenges/Challenge27CredentialsCreateAndAssignTest.kt`
- Create: `app/src/androidTest/kotlin/lava/app/challenges/Challenge31PassphraseUnlockFlowTest.kt`



Step 1: Write Challenge27 — drive Menu → Provider Credentials (or new Menu Credentials entry) → unlock with first-time setup passphrase → tap FAB → choose `USERNAME_PASSWORD` → fill name "My main creds." + username "alice" + password "pw" → Save → assert "My main creds." row appears.



Step 2: Write Challenge31 — drive Menu → Credentials → enter wrong passphrase → assert "Wrong passphrase" error → enter correct passphrase → assert list view rendered.



Step 3: Falsifiability rehearsal recorded in KDoc per Sixth Law clause 2.



Step 4: Run on the live emulator

```
./gradlew :app:connectedDebugAndroidTest -PdeviceTests=true \
-Pandroid.testInstrumentationRunnerArguments.class=lava.app.challenges.Challenge
```

Expected: 2 PASS.



Step 5: Commit with Bluff-Audit stamps for each Challenge.

Task 12: SyncOutbox module (:core:sync)

Files:

- Create: core/sync/build.gradle.kts
- Create: core/sync/src/main/kotlin/lava/sync/{SyncOutbox, SyncOutboxImpl, SyncOutboxKind}.kt
- Create: core/sync/src/main/kotlin/lava/sync/di/SyncModule.kt
- Modify: settings.gradle.kts (register module)
- Test: core/sync/src/test/kotlin/lava/sync/SyncOutboxImplTest.kt



Step 1: Write the test (Fake DAO mirroring Task 5 pattern; assert enqueue("credentials", payload) adds a row).



Step 2: Write the interface + impl — enqueue(kind: SyncOutboxKind, payload: String) writes to sync_outbox via SyncOutboxDao. kind enum: CREDENTIALS, BINDING, SYNC_TOGGLE, CLONED_PROVIDER, USER_MIRROR.



Step 3: Commit with Bluff-Audit.

Task 13: ProbeMirrorUseCase

Files:

- Create: core/domain/src/main/kotlin/lava/domain/usecase/ProbeMirrorUseCase.kt
- Test: core/domain/src/test/kotlin/lava/domain/usecase/ProbeMirrorUseCaseTest.kt



Step 1: Write the test using MockWebServer

```
package lava.domain.usecase
```

```
import kotlinx.coroutines.runBlocking
import okhttp3.OkHttpClient
import okhttp3.mockwebserver.MockResponse
import okhttp3.mockwebserver.MockWebServer
import org.junit.After
import org.junit.Assert.assertEquals
import org.junit.Before
import org.junit.Test
import java.util.concurrent.TimeUnit
```

```
class ProbeMirrorUseCaseTest {
    private lateinit var server: MockWebServer
    private val client = OkHttpClient.Builder()
        .connectTimeout(2, TimeUnit.SECONDS).readTimeout(2,
            TimeUnit.SECONDS).build()

    @Before fun setUp() { server = MockWebServer().also {
        it.start() } }
    @After fun tearDown() { server.shutdown() }

    @Test fun `200 is reachable`() = runBlocking {
        server.enqueue(MockResponse().setResponseCode(200))
        val u = ProbeMirrorUseCase(client)
        assertEquals(ProbeResult.Reachable,
            u.invoke(server.url("/").toString()))
    }

    @Test fun `5xx is reachable but unhealthy`() = runBlocking {
        server.enqueue(MockResponse().setResponseCode(503))
        val u = ProbeMirrorUseCase(client)
        assertEquals(ProbeResult.Unhealthy(503),
            u.invoke(server.url("/").toString()))
    }

    @Test fun `unreachable returns Unreachable`() = runBlocking {
        val unreachable = "https://127.255.255.254:9"
        val u = ProbeMirrorUseCase(client)
    }
```

```

        assertEquals(ProbeResult.Unreachable::class,
            u.invoke(unreachable)::class)
    }
}

```



Step 2: Write the impl

```

package lava.domain.usecase

import okhttp3.OkHttpClient
import okhttp3.Request
import java.io.IOException
import javax.inject.Inject

sealed interface ProbeResult {
    data object Reachable : ProbeResult
    data class Unhealthy(val status: Int) : ProbeResult
    data class Unreachable(val reason: String) : ProbeResult
}

class ProbeMirrorUseCase @Inject constructor(private val client:
    OkHttpClient) {
    suspend operator fun invoke(url: String): ProbeResult {
        val req = Request.Builder().url(url).head().build()
        return try {
            client.newCall(req).execute().use { resp ->
                if (resp.code in 200..399) ProbeResult.Reachable
                else ProbeResult.Unhealthy(resp.code)
            }
        } catch (e: IOException) {
            ProbeResult.Unreachable(e.message ?:
                e::class.simpleName ?: "io")
        }
    }
}

```



Step 3: Run tests + Bluff-Audit + commit.

Task 14: CloneProviderUseCase

Files:

- Create: core/domain/src/main/kotlin/lava/domain/usecase/CloneProviderUseCase.kt

- Test: core/domain/src/test/kotlin/lava/domain/usecase/CloneProviderUseCaseTest.kt



Step 1: Write the test — assert `invoke("rutracker", displayName="RuTracker EU", url="https://rutracker.eu")` writes a `ClonedProviderEntity` to the fake DAO + enqueues a `SyncOutbox` row of kind `CLONED_PROVIDER`.



Step 2: Write the impl

```
package lava.domain.usecase
```

```
import kotlinx.serialization.encodeToString
import kotlinx.serialization.json.Json
import lava.database.dao.ClonedProviderDao
import lava.database.entity.ClonedProviderEntity
import lava.sync.SyncOutbox
import lava.sync.SyncOutboxKind
import java.util.UUID
import javax.inject.Inject
```

```
class CloneProviderUseCase @Inject constructor(
    private val dao: ClonedProviderDao,
    private val outbox: SyncOutbox,
) {
    suspend operator fun invoke(sourceTrackerId: String,
        displayName: String, primaryUrl: String): String {
        val syntheticId = "${sourceTrackerId}.clone.${
            UUID.randomUUID()}"
        val entity = ClonedProviderEntity(syntheticId,
            sourceTrackerId, displayName, primaryUrl)
        dao.upsert(entity)
        outbox.enqueue(SyncOutboxKind.CLONED_PROVIDER,
            Json.encodeToString(entity))
        return syntheticId
    }
}
```



Step 3: Run + Bluff-Audit + commit.

Task 15: LavaTrackerSdk integration — union cloned providers

Files:

- Modify: `core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt` (the `listAvailableTrackers()` method).



Step 1: Locate the function — `grep -n listAvailableTrackers core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt`.



Step 2: Inject ClonedProviderDao into `LavaTrackerSdk`'s constructor (Hilt).



Step 3: Union — at the end of `listAvailableTrackers()`, fetch cloned rows + materialize them as synthetic `TrackerDescriptor` overlays (use the source descriptor as the base; override `trackerId = syntheticId, displayName, baseUrls`).



Step 4: Update test — `core/tracker/client/src/test/.../LavaTrackerSdkTest.kt`: a clone row produces a new descriptor in the list.



Step 5: Run + Bluff-Audit + commit.

Task 16: Provider Config feature module skeleton

Files:

- Create: `feature/provider_config/build.gradle.kts`
- Create: `feature/provider_config/src/main/AndroidManifest.xml`
- Create: `feature/provider_config/src/main/kotlin/lava/provider_config/{ProviderConfigScreen, ViewModel, State, Action, SideEffect, Navigation}.kt`
- Modify: `settings.gradle.kts`
- Modify: `app/src/main/kotlin/digital/vasic/lava/client/navigation/MobileNavigation.kt` (register `addProviderConfig`).



Step 1: Module + manifest + settings entry (mirror Task 10's structure).



Step 2: State/Action/SideEffect — State: provider, descriptor, boundCredential, mirrors, syncEnabled, anonymous, probeResults, cloneDialog. Action: ToggleSync, BindCredential(id), UnbindCredential, AddMirror(url), ProbeMirror(url), ToggleAnonymous, OpenClone, ConfirmClone(name, url), DismissDialogs. SideEffect: Navigate(...), ShowToast(...).



Step 3: ViewModel — wire all four DAOs (binding, sync toggle, cloned, user mirror) + LavaTrackerSdk for the provider list + CloneProviderUseCase + ProbeMirrorUseCase.



Step 4: Compile.



Step 5: Commit.

Task 17: Provider Config screen sections

Files:

- Modify: feature/provider_config/src/main/kotlin/lava/provider_config/ProviderConfigScreen.kt
- Create: feature/provider_config/src/main/kotlin/lava/provider_config/sections/{Header, SyncSection, CredentialsSection, MirrorsSection, AnonymousSection, CloneSection}.kt



Step 1: Header

```
// sections/Header.kt
@Composable
internal fun Header(state: ProviderConfigState) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        Box(Modifier.size(16.dp).background(state.color,
            CircleShape))
        Spacer(Modifier.width(AppTheme.spaces.medium))
        Column(Modifier.weight(1f)) {
            HeadlineSmall(text = state.displayName)
        }
    }
}
```

```

        BodySmall(text = state.descriptor?.trackerId ?: "")
    }
    ConnectionStatusIcon(state.connectionStatus)
}
}

```



Step 2: SyncSection — `SwitchRow(label = "Sync this provider", checked = state.syncEnabled, onToggle = { onAction(ToggleSync) })`.



Step 3: CredentialsSection — shows bound credential by display name OR "None — anonymous"; two buttons "Assign existing..." / "Create new...". "Assign existing..." opens a `ModalBottomSheet` listing all `CredentialsEntry.displayName`; tap one → `BindCredential(id)`. "Create new..." launches the create-credentials sheet from `:feature:credentials_manager` (cross-feature reuse — extract the sheet composable to `:feature:credentials_manager` public API).



Step 4: MirrorsSection — `Column` of `MirrorRow(host, status, isPrimary, isUserAdded, onProbe, onRemove)` for every descriptor mirror + every `UserMirror`. Add-new text field at the bottom.



Step 5: AnonymousSection — guarded by `descriptor.supportsAnonymous`; `SwitchRow(label = "Anonymous", state.anonymous, onToggle = { onAction(ToggleAnonymous) })`.



Step 6: CloneSection — `OutlinedButton(text = "Clone provider...")` → opens `AlertDialog` with two `TextFields` (display name + primary URL) + `Confirm/Cancel`.



Step 7: Wire screen layout — `Scaffold(topBar = ...)`
`{ Column(verticalScroll(rememberScrollState())) { Header() / SyncSection() / CredentialsSection() / MirrorsSection() / AnonymousSection() / CloneSection() } }`. Per §6.Q no `LazyColumn` inside `verticalScroll`.



Step 8: Commit.

Task 18: Menu wiring — provider row → ProviderConfig

Files:

- Modify: `feature/menu/src/main/kotlin/lava/menu/MenuScreen.kt:383-424` (ProviderRow) and the Menu's Provider Credentials entry on line ~204.



Step 1: Make the surface clickable

Change the `Surface(modifier = Modifier.fillMaxWidth(), shape = ..., color = ...)` to take an `onClick` that triggers `MenuAction.OpenProviderConfig(provider.providerId)`.



Step 2: Move sign-out — the existing `IconButton` for sign-out moves to a trailing affordance. Sign-out should now require a confirmation dialog (per the existing `MenuAction.ConfirmSignOut` flow which is already in place).



Step 3: Update Menu's "Provider Credentials" entry to route to the new `:feature:credentials_manager` instead of the legacy per-provider screen.



Step 4: Update MenuViewModel to add `OpenProviderConfig(providerId)` action emitting a `MenuSideEffect.OpenProviderConfig(providerId)`.



Step 5: Wire navigation in `MobileNavigation.kt` to handle the new side effect via `navController.navigate(openProviderConfig(providerId))`.



Step 6: Commit.

Task 19: Challenges 28-30 + run on emulator

Files:

- Create: `app/src/androidTest/kotlin/lava/app/challenges/Challenge28PerProviderSyncToggleTest.kt`

- Create: app/src/androidTest/kotlin/lava/app/challenges/Challenge29AddCustomMirrorTest.kt
- Create: app/src/androidTest/kotlin/lava/app/challenges/Challenge30CloneProviderTest.kt



Step 1: Challenge28 — drive Menu → tap provider row → assert provider config screen renders → toggle "Sync this provider" OFF → press back → tap provider row again → assert switch is still OFF → toggle ON → assert sync_outbox table has at least one row with kind = "SYNC_TOGGLE".



Step 2: Challenge29 — drive Menu → tap provider row → in MirrorsSection enter "rutracker.example.org" + Add → assert new MirrorRow appears in the list → tap Probe → assert ProbeResult.Unreachable styling.



Step 3: Challenge30 — drive Menu → tap provider row → tap "Clone provider..." → fill "RuTracker EU" + https://rutracker.eu → Confirm → assert Menu now shows "RuTracker EU" row alongside the original.



Step 4: KDoc each with mutation / observed-failure / revert per \$6.J/\$6.N.



Step 5: Run on emulator

```
export ANDROID_SERIAL=emulator-5554
./gradlew :app:connectedDebugAndroidTest -PdeviceTests=true \
  -Pandroid.testInstrumentationRunnerArguments.class=\
  lava.app.challenges.Challenge28PerProviderSyncToggleTest,\
  lava.app.challenges.Challenge29AddCustomMirrorTest,\
  lava.app.challenges.Challenge30CloneProviderTest
```

Expected: 3 PASS.



Step 6: Commit with three Bluff-Audit stamps in the body.

Task 20: Distribute — version bump + CHANGELOG + snapshot + pepper rotation + Firebase

Files:

- Modify: `app/build.gradle.kts:28-29` — `versionCode 1036` → `1037`, `versionName 1.2.16` → `1.2.17`.
- Modify: `lava-api-go/internal/version/version.go` — `Code 2305` → `2306`, `Name 2.3.5` → `2.3.6`.
- Modify: `CHANGELOG.md` (prepend new entry).
- Create: `.lava-ci-evidence/distribute-changelog/firebase-app-distribution/1.2.17-1037.md`.
- Modify: `.env` — rotate `LAVA_AUTH_OBFUSCATION_PEPPER`, bump `LAVA_AUTH_CURRENT_CLIENT_NAME`, append to `LAVA_AUTH_ACTIVE_CLIENTS`.
- Modify: `.lava-ci-evidence/distribute-changelog/firebase-app-distribution/last-version` → `1036` (so the script accepts the bump).



Step 1: Version bumps (two edits as above).



Step 2: CHANGELOG entry — new section at top documenting Phase A + B additions, Challenges 27-31, honest-stub on the sync uploader (Phase E owed), version bumps.



Step 3: Per-version snapshot — copy the relevant subset of the CHANGELOG section into the snapshot file.



Step 4: Pepper rotation

```
NEW_PEPPER=$(openssl rand -base64 32)
NEW_UUID=$(uuidgen)
# ... same python3 inline rotate as v1.2.16 (already documented in
# previous release commits – copy the pattern verbatim).
```



Step 5: Rebuild APKs

```
./gradlew :app:assembleDebug :app:assembleRelease --parallel
```

Expected: BUILD SUCCESSFUL. Two APKs in `app/build/outputs/apk/{debug,release}/`.



Step 6: Copy to releases/

```
mkdir -p releases/1.2.17/android-{debug,release}  
cp app/build/outputs/apk/debug/app-debug.apk releases/1.2.17/  
    android-debug/  
cp app/build/outputs/apk/release/app-release.apk releases/1.2.17/  
    android-release/
```



Step 7: Restart Go API at v2.3.6

```
bash stop.sh  
bash start.sh  
sleep 8  
curl -sSk https://localhost:8443/health
```

Expected: {"status":"alive"}.



Step 8: Reset last-version

```
echo 1036 > .lava-ci-evidence/distribute-changelog/firebase-app-  
distribution/last-version
```



Step 9: Run firebase-distribute.sh

```
bash scripts/firebase-distribute.sh
```

Expected: \$6.P gates pass, two releases uploaded, last-version recorded as 1037.



Step 10: Commit + push with full release commit message (mirror the v1.2.16 commit pattern, include Bluff-Audit stamps for any test changes in this final commit).

```
for r in github gitlab; do git push "$r" master; done
```

Task 21: CONTINUATION.md update

Files:

- Modify: docs/CONTINUATION.md \$0 (Last updated).



Step 1: Edit \$0 to reflect v1.2.17-1037 distribute + SP-4 Phase A+B complete + remaining phases C-H still scoped.



Step 2: Commit

```
git commit -m "docs(continuation): SP-4 Phase A+B complete;  
v1.2.17-1037 distributed"  
for r in github gitlab; do git push "$r" master; done
```

Self-Review

1. Spec coverage:

- Phase A domain model → Task 1 ✓
- AES-256-GCM + PBKDF2 + verifier → Task 2 ✓
- Room schema bump v8→v9 + migrations → Tasks 3-4 ✓
- CredentialsEntryRepository → Task 5 ✓
- ProviderCredentialBinding → Task 6 ✓
- Passphrase session cache → Task 7 ✓
- PassphraseManager + Storage → Task 8 ✓
- Hilt wiring → Task 9 ✓
- Credentials Manager UI → Task 10 ✓
- Challenge27 + Challenge31 → Task 11 ✓
- SyncOutbox → Task 12 ✓
- ProbeMirrorUseCase → Task 13 ✓
- CloneProviderUseCase → Task 14 ✓
- LavaTrackerSdk integration → Task 15 ✓
- Provider Config feature module → Tasks 16-17 ✓
- Menu wiring → Task 18 ✓
- Challenges 28-30 → Task 19 ✓
- Distribute → Task 20 ✓
- CONTINUATION update → Task 21 ✓

2. Placeholder scan: No TBD/TODO/FIXME/placeholder text in task bodies. ✓ (legacy migration of Phase-11 per-provider creds was scoped in the spec but is deferred — flagged as Phase A-2 in CONTINUATION; not a placeholder in this plan).

3. Type consistency: CredentialsEntry, CredentialSecret, CredentialType consistent. ProviderConfigState/Action/SideEffect referenced consistently. DAOs match entity names. ✓

4. Subagent decomposition mapping:

- S-A1 (foundation): Tasks 1-9.
- S-A2 (UI): Tasks 10-11.

- S-Sync: Tasks 12-15.
 - S-B1 (screen scaffold): Tasks 16-17.
 - S-B2 (Menu wiring): Task 18.
 - S-T1 (Challenges + emulator): Task 19.
 - S-Release: Tasks 20-21.
-

Execution Handoff

Plan complete and saved. Two execution options:

1. Subagent-Driven (recommended) — fresh subagent per task group (S-A1, S-A2, S-Sync, S-B1, S-B2, S-T1, S-Release), main agent reviews between groups. Best for plan this size.

2. Inline Execution — execute tasks sequentially in this session, batch commits with checkpoint reviews.